

ECS401: Unit 3 Lab Progress Test

FEEDBACK: Answers and Explanations

Work hard and you will eventually be able to program.

If you take it seriously, this sheet will help improve your understanding and the lab tutors and peer tutors can give you more help if you need it.

1. Write a Java statement that declares a variable called `famous` and initialises it to value `true`.

CORRECT ANSWER:

```
boolean famous = true;
```

EXPLANATION:

The variable has to hold `true` and `false` values so that means it is a boolean variable that is needed. The type `boolean` is used to mean the type of `true` and `false` values but otherwise the declaration is no different to declaring and initialising an integer or string variable.

For questions 2, 3 and 4 first consider the following fragment of code and complete the dry run table in each case:

```
if (superhero == 1)                // L1
{ System.out.println("Batman"); } // L2
else
{ System.out.println("Joker"); }   // L3
```

2. Complete the dry run table. What is printed to the screen if variable `superhero` holds the value 0 immediately before the above fragment is executed?

DRY RUN TABLE:

Line	superhero	superhero == 1 ?	printed to screen
-	0		
L1		0 == 1 ? false	
L3			Joker

CORRECT ANSWER:

Joker

EXPLANATION:

The test that is done is asking the question “True or false: Is superhero equal to 1?”. The value of `superhero` is 0. This means the question is really “True or false: Is 0 equal to 1?”. That is false. That means the else branch of the if statement is executed, so `Joker` is printed. The `Batman` instruction is ignored.

3. Complete the dry run table. What is printed to the screen if variable `superhero` holds the value 1 immediately before the above fragment is executed?

DRY RUN TABLE:

Line	superhero	superhero == 1 ?	printed to screen
-	1		
L1		1 == 1 ? true	
L2			Batman

CORRECT ANSWER:

Batman

EXPLANATION:

The test that is done is asking the question “True or false: Is superhero equal to 1?”. The value of `superhero` is 1 this time. This means the question is really “True or false: Is 1 equal to 1?”. That is true. That means the first branch of the if statement is executed, so `Batman` is printed. The `Joker` instruction is ignored this time.

4. Complete the dry run table. What is printed to the screen if variable `superhero` holds the value 2 immediately before the above fragment is executed?

DRY RUN TABLE:

Line	superhero	superhero == 1 ?	printed to screen
-	2		
L1		2 == 1 ? false	
L3			Joker

CORRECT ANSWER:

Joker

EXPLANATION:

This is no different to what happens in question two. The value 2 is not equal to 1 so the else case is executed.

5. What boolean expression would be used to test if the String variable `actress` holds value: "Keira Knightley"?

CORRECT ANSWER:

```
actress.equals("Keira Knightley")
```

EXPLANATION:

This is about comparing strings. You must use the `.equals()` method to do this.

If you thought the answer was `(actress == "Keira Knightley")` then you are confused over the two kinds of equality comparison. `==` is used to compare simple things like integers and characters only, not strings.

If you thought the answer was `(actress = "Keira Knightley")` then you are confused with assignment. A single `=` symbol means assign so this would try and store the string "Keira Knightley" into the variable `actress`.

For questions 6, 7 and 8, consider the following fragment of code and complete the dry run table in each case:

```
if (singer > 2)                                // L1
{ System.out.println("Duffy"); }               // L2
else if (singer == 3)                          // L3
{ System.out.println("Madonna"); }             // L4
else
{ System.out.println("KT Tunstall"); }         // L5
```

6. Complete the dry run table. What is printed to the screen if variable `singer` holds the value 2 immediately before the above fragment is executed?

DRY RUN TABLE:

Line	singer	singer > 2 ?	singer == 3 ?	printed to screen
-	2			
L1		2 > 2 ? false		
L3			2 == 3 ? false	
L5				KT Tunstall

CORRECT ANSWER:

KT Tunstall

EXPLANATION:

The first test is asking if `singer` is greater than 2. 2 is not greater than itself so the test is `false` and it skips to the first `else` part where there is another test. That tests if `singer` is equal to 3. It isn't so it skips to the final `else` case and executes that, so `KT Tunstall` appears on the screen.

7. Complete the dry run table. What is printed to the screen if variable `singer` holds the value 3 immediately before the above fragment is executed?

DRY RUN TABLE:

Line	singer	singer > 2 ?	singer == 3 ?	printed to screen
-	3			
L1		3 > 2 ? true		
L2				Duffy

CORRECT ANSWER:

Duffy

EXPLANATION:

The first test is asking if `singer` is greater than 2. 3 is greater than 2 so the test is `true` and `Duffy` is printed. It then skips beyond all the `elses` so the others aren't printed.

If you thought that `Madonna` would be printed you have misunderstood how an `if-then-else if ... staircase` like this works. It goes down the series of tests after the 'if's, checking the test until it finds the first test that is `true`. That bit of code only is executed. The others are skipped over. It does not just look for the best match – tests are either `true` or `false` so it makes no sense to think of a 'best' match!

8. Complete the dry run table. What is printed to the screen if variable `singer` holds the value 4 immediately before the above fragment is executed?

DRY RUN TABLE:

Line	singer	singer > 2 ?	singer == 3 ?	printed to screen
-	4			
L1		4 > 2 ? true		
L2				Duffy

CORRECT ANSWER:

Duffy

EXPLANATION:

As in the previous case, 4 is greater than 2 so the first test is done and nothing else.

Note this code will never print Madonna!

Terminology

9. **Expressions** in a program are parts of a program that evaluate to give an answer. Highlight/underline all the expressions in the following fragment of code.

```
int superhero = 10;  
int supervillian = superhero - 5;  
boolean saved = true;  
  
if (superhero > supervillian)  
{  
    System.out.println("Saved");  
    supervillian = supervillian - 5;  
    saved = true;  
}  
else  
{  
    System.out.println("Oh no");  
    supervillian = supervillian - 5;  
    saved = ! saved;  
}
```

10. A **boolean expression** is one that evaluates to a boolean (true or false) value. Highlight/underline all the boolean expressions in the following fragment of code.

```
int superhero = 10;  
int supervillian = superhero - 5;  
boolean saved = true;  
  
if (superhero > supervillian)  
{  
    System.out.println("Saved");  
}
```

```

        supervillian = supervillian - 5;
        saved = true;
    }
    else
    {
        System.out.println("Oh no");
        supervillian = supervillian - 5;
        saved = ! saved;
    }
}

```

11. An **assignment** stores a value in a variable. Its right hand side is an expression. Highlight/underline all the assignments in the following fragment of code.

```

int superhero = 10;
int supervillian = superhero - 5;
boolean saved = true;

if (superhero > supervillian)
{
    System.out.println("Saved");
    supervillian = supervillian - 5;
}
else
{
    saved = ! saved;
}

```

12. An **accessor** method is a method that either returns the value stored in a specific field of a record or stores a new value in a specific field of a record.

Write two accessor methods: one that returns the value from the `destination` field of the record below, and one that changes it to a given value, returning the new record.

```

class Train
{
    String source;
    String destination;
}

```

CORRECT ANSWER:

```
public static String getTrainDestination(Train t)
{
    return t.destination;
} // END getTrainDestination

public static Train setTrainDestination(Train t, String newdest)
{
    t.destination = newdest;
    return t;
} // END setTrainDestination
```